

FRED TALKS

27th September 2026

CONTENTS

I got an email about resistance

SEANGOEDECKE.COM RSS FEED · 1351 WORDS

Getting Real About Distributed System Reliability

BOREDANDROID · 2792 WORDS

Please for the love of all that is holy: Stop writing long boring
Titles

SWYX · 1431 WORDS

the senior engineer death spiral

SUNIL PAI · 959 WORDS

I got an email about resistance

SEANGOEDECKE.COM RSS FEED · 09 AUG 2026 · [SOURCE](#)

This will be kind of an unusual post. I got a recent email about my writing that I thought was such a good articulation of one common criticism that I'd like to share it (and my response) in full.

Here's the email, from William Murray¹:

Hey Sean,

I have enjoyed your writing but your recent essays frustrate me.

You say that getting paid for deep thinking in software is coming to an end. You even admit that it makes you sad. But in the name of "usefulness" you refuse to rock the boat. The way I see it, if you are right there are only two reasonable responses, pursue other work or resist. You present your elegiac approach as mature / pragmatic / realistic. I'd call it complicit. You know when Willy Wonka says,

There's no earthly way of knowing Which direction we are going There's no knowing where we're rowing Or which way the river's flowing Is it raining, is it snowing? Is a hurricane a-blowing? — uh! Not a speck of light is showing So the danger must be growing Are the fires of Hell a-glowing? Is the grisly reaper mowing? Yes! The danger must be growing For the rowers keep on rowing And they're certainly not showing Any signs that they are slowing!

And the audience is thinking, "isn't Wonka kind of in control of this situation?" You remind me of Wonka². You write like a passenger on a crazy train going who-knows-where! But you are an agent. You are in control of your life! Either admit that you actually like where the crazy train is probabilistically going or get off at the next stop.

You have a lot of reach and you are using it for... what exactly? Showing off how pragmatic you are by being more black pill'd than the next guy? Broadcasting your resignation to the unstoppable trends of technology is a waste of a voice.

You may find this argument absurd, but I don't so I'll make it. This is a very important time in history. I hope humanity survives and continues to grow exponentially. In that case the supply of historical people will stay fixed while the supply of contemporary people will keep growing. There will come a day where for every 2026 staff software engineer there are dozens of

historians specializing in 2020s era software engineering culture. It's plausible that your essays will be remembered for all of time and your actions will be judged by history. Do you want future humans to see you as a rationalizing careerist or something cooler?

Sorry for the haranguing email from a stranger, I'm sending it for the small chance that it awakens something in you. If I'm way off I'm sorry.

And here's my response:

Hey William, thanks for emailing.

I wish everyone who thought this way emailed me so I could think harder about this kind of position. Despite what my writing might suggest, I do in fact think a lot about it. Let me see if I can explain my position in a way you'll find satisfying.

I agree that this is an important time in history. For programmers, I think of it as analogous to the Industrial Revolution in England: we are a group of high-status craftspeople who find ourselves alternately threatened and empowered by automation. The developments today, as then, obviously have far-reaching implications — but what those implications are is very non-obvious. Would a framework-knitter in the early 1800s have been able to predict the ramifications of the stocking frame on the world of today? What should they have done about it, in order to be kindly judged by history?

Well, we know what many of them did do. They shot factory-owners, smashed machines, burned down the factories — in some places delaying the spread of automation; in other places encouraging it — prompting a crackdown that saw tens of thousands of British soldiers occupying British counties in what was clearly a police state. History judges the Luddites kindly for this. Does that mean it worked?

I don't care about the judgment of history. They'll think what they want. What I care about is **the people in my industry who don't know what to do**. I get hundreds of emails from junior and mid-level (and other) engineers who say "I'm scared, I don't know the rules post-2021, thank you for helping me keep my head down and keep my job". That's why I write the way I write. I have seen lots of idealistic engineers stick their necks out, and post-ZIRP those necks often get cut off. That's a damn shame.

I think it's morally wrong that so many engineers — either in safe sinecures in big tech or literally retired — seem to be trying to foment a second Luddite revolution. Many of their readers will be experienced enough to handle it sensibly, but not all. Every “AI is fascist, stand up and resist!” post that goes viral ruins some poor idealistic junior’s career³. Someone needs to be out there saying “hey, if you do X it’s going to have consequence Y”. I hope that’s me.

Of course this is complicit, or anti-revolutionary, or whatever you like. But if I were a textiles worker in 1810s England, I would not be telling my friends and loved ones “it’s time to fight, let’s go smash up the factories for Ned Ludd!“. I would be telling them that this was the most dangerous time in the industry (perhaps ever), and that they ought to be very damn careful so they don’t get shot, or arrested, or hanged. If I then went and told a few hundred thousand strangers the opposite, I would be a hypocrite.

Anyway, I do take this view seriously — seriously enough to vehemently disagree, at least — which I hope you’ll find better than me just shrugging it off. I do accept the existence of some kind of line: I think Industrial-Revolution-collaborating was OK but Nazi-collaborating wasn’t, for instance. But in the current situation, the way I’m spending “my voice” is to try and prevent the most vulnerable of my colleagues from making career-ruining mistakes.

Sean

In this blog, I try to encourage people to work with the system, to learn its rules, and to try and exert influence safely from a position of power, instead of openly picking fights with their employers. I’ve written and read about the Luddites before, but I remain deeply ambivalent about the movement itself, and about modern-day attempts to resurrect it in service of anti-AI activism.

I want to explicitly thank Murray for writing such a thoughtful email, and being willing for me to publish it on the blog.

1. Shared with permission, of course. I’ve lightly edited both Murray’s email and mine for typos.

↩

2. I didn’t pick up this point in my reply, but I’ll briefly mention it here: Wonka is in control because he owns the factory and the rowers in question are *his employees*. I don’t think the position of any engineer (or of almost any manager) is like that.

↩

3. In hindsight, I think this is a little overstated, but it does happen and causes a lot of needless suffering.

↩

Getting Real About Distributed System Reliability

BOREDANDROID · 01 MAR 2026 · [SOURCE](#)

There is a lot of hype around distributed data systems, some of it justified. It's true that the internet has centralized a lot of computation onto services like Google, Facebook, Twitter, LinkedIn (my own employer), and other large web sites. It's true that this centralization puts a lot of pressure on system scalability for these companies. It's true that incremental and horizontal scalability is a *deep* feature that requires redesign from the ground up and can't be added incrementally to existing products. It's true that, if properly designed, these systems can be run with no *planned* downtime or maintenance intervals in a way that traditional storage systems make harder. It's also true that software that is explicitly designed to deal with machine failures is a very different thing from traditional infrastructure. All of these properties are critical to large web companies, and are what drove the adoption of horizontally scalable systems like [Hadoop](#), [Cassandra](#), [Voldemort](#), etc. I was the original author of Voldemort and have worked on distributed infrastructure for the last four years or so. So in-so-far as there is a "big data" debate, I am firmly in the "pro-" camp. But one thing you often hear is that this kind of software is more reliable than the traditional alternatives it replaces, and this just isn't true. It is time people talked honestly about this.

You hear this assumption of reliability everywhere. Now that scalable data infrastructure has a marketing presence, it has really gotten bad. Hadoop or Cassandra or what-have-you can tolerate machine failures then they must be unbreakable right? Wrong.

Where does this come from? Distributed systems need to partition data or state up over lots of machines to scale. Adding machines increases the probability that some machine will fail, and to address this these systems typically have some kind of replicas or other redundancy to tolerate failures. The core argument that gets used for these systems is that if a single machine has probability P of failure, and if the software can replicate data N times to survive $N-1$ failures, and if the machines fail *independently*, then the probability of losing a particular piece of data

must be P^N . So for any desired reliability R and any single-node failure probability P you can pick some replication N so that $P^N < R$. This argument is the core motivation behind most variations on replication and fault tolerance in distributed systems. It is true that without this property the system would be hopelessly unreliable as it grew. But this leads people to believe that distributed software is somehow innately reliable, which unfortunately is utter hogwash.

Where is the flaw in the reasoning? Is it the dreaded Hadoop single points of failure? No, it is far more fundamental than that: the problem is the assumption that failures are independent. Surely no belief could possibly be more counter to our own experience or just common sense than believing that there is no correlation between failures of machines in a cluster. You take a bunch of pieces of identical hardware, run them on the same network gear and power systems, have the same people run and manage and configure them, and run the same (buggy) software on all of them. It would be incredibly unlikely that the failures on these machines would be independent of one another in the probabilistic sense that motivates a lot of distributed infrastructure. If you see a bug on one machine, the same bug is on all the machines. When you push bad config, it is usually game over no matter how many machines you push it to.

P^N is an upper bound on reliability but one that you could never, never approach in practice. For example Google has a fantastic paper that gives empirical numbers on system failures in Bigtable and GFS and reports empirical data on groups of failures that show rates several orders of magnitude higher than the independence assumption would predict. This is what one of the best system and operations teams in the world can get: your numbers may be far worse.

The actual reliability of your system depends largely on how bug free it is, how good you are at monitoring it, and how well you have protected against the myriad issues and problems it has. This isn't any different from traditional systems, except that the new software is far less mature. I don't mean this disparagingly, I work in this area, it is just a fact. Maturity comes with time and usage and effort. This software hasn't been around for as long as MySQL or Oracle, and worse, the expertise to run it reliably is much less common. MySQL and Oracle administrators are plentiful, but folks experience with, say, serious production Zookeeper operations knowledge are much more rare.

Kernel filesystem engineers say it takes about a decade for a new filesystem to go from concept to maturity. I am not sure these systems will be mature much faster—they are not easier systems to build and the fundamental design space is much less well explored. This doesn't mean they won't be *useful* sooner, especially in domains where they solve a pressing need and are approached with an appropriate amount of caution, but they are not yet by any means a mature technology.

Part of the difficulty is that distributed system software is actually quite complicated in comparison to single-server code. Code that deals with failure cases and is “cluster aware” is extremely tricky to get right. The root of the problem is that dealing with failures effectively explodes the possible state space that needs testing and validation. For example it doesn’t even make sense to expect a single-node database to be fast if its disk system suddenly gets really slow (how could it), but a distributed system does need to carry on in the presence of single degraded machine because it has some many machines, one is sure to be degraded. These kind of “semi-failures” are common and very hard to deal with. Correctly testing these kinds of issues in a realistic setting is brutally hard and the newer generation of software doesn’t have anything like the QA processes its more mature predecessors had. (If you get a chance get someone who has worked at Oracle to describe to you what kind of testing they do to a line of code that goes into their database before it gets shipped to customers). As a result there are a lot of bugs. And of course these bugs are on all the machines, so they absolutely happen together.

Likewise distributed systems typically require more configuration and more complex configuration because they need to be cluster aware, deal with timeouts, etc. This configuration is, of course, shared; and this creates yet another opportunity to bring everything to its knees.

And finally these systems usually want lots of machines. And no matter how good you are, some part of operational difficulty always scales with the number of machines.

Let’s discuss some real issues. We had a bug in [Kafka](#) recently that lead to the server incorrectly interpreting a corrupt request as a corrupt log, and shutting itself down to avoid appending to a corrupt log. Single machine log corruption is the kind of thing that should happen due to a disk error, and bringing down the corrupt node is the right behavior—it shouldn’t happen on all the machines at the same time unless all the disks fail at once. But since this was due to corrupt *requests*, and since we had one client that sent corrupt requests, it was able to sequentially bring down all the servers. Oops. Another example is [this Linux bug](#) which causes the system to crash after ~200 days of uptime. Since machines are commonly restarted sequentially this lead to a situation where a large percentage of machines went hard down one after another. Likewise any memory management problems—either leaks or GC problems—tend to happen everywhere at once or not at all. Some companies do public post-mortums for major failures and these are a real wealth of failures in systems that aren’t supposed to fail. [This paper](#) has an excellent summary of HDFS availability at Yahoo—they note how few of the problems are of the kind that high availability for the namenode would solve. This list could go on, but you get the idea.

I have come around to the view that the real core difficulty of these systems is operations, not architecture or design. Both are important but good operations can often work around the limitations of bad (or incomplete) software, but good software cannot run reliably with bad operations. This is quite different from the view of unbreakable, self-healing, self-operating

systems that I see being pitched by the more enthusiastic NoSQL hypesters. Worse yet, you can't easily buy good operations in the same way you can buy good software—you might be able to hire good people (if you can find them) but this is more than just people; it is practices, monitoring systems, configuration management, etc.

These difficulties are one of the core barriers to adoption for distributed data infrastructure. LinkedIn and other companies that have a deep interest in doing creative things with data have taken on the burden of building this kind of expertise in-house—we employ committers on Hadoop and other open source projects on both our engineering and operations team, and have done a lot of from-scratch development in this space where there was gaps. This makes it feasible to take full advantage of an admittedly valuable but immature set of technologies, and let's us build products we couldn't otherwise—but this kind of investment only makes sense at a certain size and scale. It may be too high a cost for small startups or companies outside the web space trying to bootstrap this kind of knowledge inside a more traditional IT organization.

This is why people should be excited about things like Amazon's DynamoDB. When DynamoDB was released, the company DataStax that supports and leads development on Cassandra released a feature comparison checklist. The checklist was unfair in many ways (as these kinds of vendor comparisons usually are), but the biggest thing missing in the comparison is that *you* don't run DynamoDB, Amazon does. That is a huge, huge difference. Amazon is good at this stuff, and has shown that they can (usually) support massively multi-tenant operations with reasonable SLAs, *in practice*.

I really think there is really only one thing to talk about with respect to reliability: continuous hours of successful production operations. That's it. In some ways the most obvious thing, but not typically what you hear when people talk about these systems. I will believe the system can tolerate (some) failures when I see it tolerate those failures; I believe it can run for a year without downtime when I see it run for a year without downtime. I call this empirical reliability (as opposed to theoretical reliability). And getting good empirical reliability is really, really hard. These systems end up being large hunks of monitoring, tests, and operational procedures with a teeny little distributed system strapped to the back.

You see this showing up in discussions of the CAP theorem all the time. The CAP theorem is a useful thing, but it applies more to system design than system implementations. A design is simple enough that you can maybe prove it provides consistency or tolerates partition failures under some assumptions. This is a useful lens to look at system designs. You can't hope to do this kind of proof with an actual system implementation, the thing you run. The difficulty of building these things means it is really unthinkable that these systems are, in actual reality, either consistent, available, *or* partition tolerant—they certainly all have numerous bugs that will break

each of these guarantees. I really like [this paper](#) that takes the approach of actually trying to calculate the observed consistency of eventually consistent systems—they seem to do it via simulation rather than measurement, which is unfortunate, but the idea is great.

It isn't that system design is meaningless, it is worth discussing the system design as it does act as a kind of limiting factor on certain aspects of reliability and performance as the implementation matures and improves, but don't take it too seriously as guaranteeing *anything*.

So why isn't this kind of empirical measurement more talked about? I don't know. My pet theory is it has to do with the somewhat rigid and deductive mindset of classical computer science. This is inherited from pure math, and conflicts with the attitude in scientific disciplines. It leads to a preference for starting with axioms, and then proving various properties that follow from these axioms. This world view doesn't embrace the kind of empirical measurements you would expect to justify claims about reality (for another example see [this great blog post](#) on programming language productivity claims in programming language research). But that is getting off topic. Suffice it to say, when making predictions about how a system will work in the real world I believe in measurements of reality a lot more than arguments from first-principles.

I think we should insist on a little more rigor and empiricism in this area.

I would love to see claims in academic publication around practicality or reliability justified in the same way we justify performance claims—by doing it. I would be a lot more likely to believe an academic distributed system was practically feasible if it was run continuously under load for a year successfully and if information was reported on failures and outages. Maybe that isn't feasible for an academic project, but few other allegedly scientific academic disciplines can get away with making claims about reality without evidence.

More broadly I would like to see more systems whose *operation* is verifiable. Many systems have the ability to log out information about their state in a way that makes programmatic checking of various invariants and guarantees possible (such as consistency or availability). An example of such an invariant for a messaging system is that all the messages sent to it are received by all the subscribers. We actually measure the truth of this statement in real-time in production for our Kafka data pipeline for all 450 topics and all their subscribers. The number of corner-cases one uncovers with this kind of check, run through a few hundred use cases and a few billion messages per day is truly astounding. I think this is a special case of a broad class of verification that can be done on the running system that goes far far deeper than what is traditionally considered either monitoring or testing. Call it unit testing in production, or self-proving systems, or next generation monitoring, or whatever, but I think this kind of deep verification is something that makes turning the theoretical claims a design makes into measured properties of the real running system.

Likewise if you have a “NoSQL vendor” I think it is reasonable to ask them to provide hard information on customer outages. They don’t need to tell you who the customer is, but they should let you know the observed real-life distribution of MTTF and MTTR they are able to achieve, not just highlight one or two happy cases. Make sure you understand how they measure this, do they have automated test load that runs or just wait for people to complain? This is a reasonable thing for people paying for a service to ask for. To a certain extent if they provide this kind of empirical data it isn’t clear why you should even *care* what their architecture is beyond intellectual curiosity.

Distributed systems solve a number of key problems at the heart of scaling large websites. I absolutely think this is going to become the default approach to handling state in internet application development. But no one benefits from the kind of irrational exuberance that currently surrounds the “big data” or nosql systems communities. This area is experiencing a boom—but nothing takes us from boom to bust like unrealistic hype and broken promises. We should be a little more honest about where these systems already shine and where they are still maturing.

Postscript: This blog seems to have resurfaced 7 years later. I actually still agree with much of it, though the dilemma is much less important. Distributed systems operations is hard, but you can now avoid it by just getting your distributed systems as a service. For example, Confluent offers Kafka as a service with a contractual uptime SLA. Ops remains hard, but for distributed data systems you can make the people who wrote the software deal with the problem, and focus your time on your special sauce.

Please for the love of all that is holy: Stop writing long boring Titles

SWYX · 30 MAR 2026 · [SOURCE](#)

People code switch. Every ethnic minority has lived experience of this. When you talk to me in Singapore, I sound very different than when I’m in San Francisco. And the way you talk at work is probably very different from how you talk to your kids and how you talk to your lover.

I've been dealing with a particularly virulent form of code switching in the simple form of titling blogposts and talks. When you ask the writer face to face what their thing is about, they can give you a perfectly human explanation. And then when they send you their piece, it's all ***"Leveraging GenAI with Robust Data Foundations to Transform Businesses"***. I've left my body by the time I read the first word.

My hope in writing this piece is to **put an end to bad titles**.

If I sent you this guide, it is not an insult. You just don't have good title game. That's ok. You're great at what you do, title game isn't your job. I'm the guy that wrote How To Name Things and Two Words. I sent you this guide ***because*** your shitty title is the main thing stopping your readers from getting to the good part. If you didn't have anything good, I wouldn't bother with a lost cause.

First...

Towards Leveraging Corporate Speak For Maximum Enterprise Synergy

DO NOT:

- Start with "Leveraging" or "Towards" or "Navigating" or "Empowering"

Yeah just stop. There's no rescuing this when you start there. Just delete meaningless corporate weasel words. Anything >3 syllables is a red flag.

Make It Quotable

If you do not try to make your title roll off the tongue, it probably won't.

Literally imagine telling your friend about your talk. Telling a new acquaintance to google your blogpost. Having a fan of yours cite it back to you. How much do they stumble? Yeah. This has all happened to me.

There's an extremely high correlation between what are considered the best talks and the quotability of titles. Simple Made Easy. Inventing on Principle. Our list of best demos/pitches of all time.

Resist colons. It lacks confidence. Why Use Lot Word: ~~When Few Word Do Trick~~? You're not High School Musical: The Musical: The Series.

Put the SEO keyword vomit in the description.

Titles are for humans.

Prescribe A Strong Opinion

Opinions are one of the most quotable things.

But more importantly, a good Opinion Piece is load bearing because it is eminently forwardable, and therefore viral and impactful. Your Job To Be Done is to be the definitive, conclusive, piece on that Opinion. Stake out territory in Opinion Space, list out all the arguments for, counter all the arguments against, provide datapoints, and watch that go further than anything you've ever done.

How to know what's a good opinion? **Three Strikes rule:** If you repeat yourself three times, ship it. Even if you're just quoting others, its fine, credit them and make it your own.

Mental Models and Frameworks are more complex opinions that allow others to scaffold their thought around you. My Radiating Circles and Measuring Developer Relations and Third Age of JavaScript did this.

Opinions contrary to an established opinion are the hardest to do well and the best when correct. What Clayton Christensen Got Wrong made Ben Thompson's career. Rise of AI Engineer countered the prevailing Prompt Engineer -and- GPU Rich Startup/ML Engineer narrative at the time, with room to grow.

Curiosity Gap Dynamics

Marketers know the power of the Curiosity Gap. Here is where we toy closest with clickbait. I'll be honest that I wasn't even sure if I wanted to give it its own subheading, but its a very important concept that you should at least be aware exists, even if you're going to ignore it.

Strong enough opinions create their own curiosity gaps; you won't have to try very hard to make it interesting if you got my attention in 4 words anyway.

Curiosity gaps done poorly can lead to "burying the lede", which make your talk less Quotable and you also don't want to do that.

Simplest way to put it is; if the most interesting thing about your content is your title, and what people take away after having read your post matches what they would have predicted after reading your title, then you've probably said too much and could cut.

Play Madlibs

You don't have to invent your title from scratch (nor should you always do what I'm about to suggest). Here's some templates that work well:

- {*BENEFIT EVERYONE WANTS*} **without** {*DOWNSIDE EVERYBODY EXPECTS*}
 - e.g. How to Get Rich (Without Getting Lucky), How to Market Yourself (Without Being A Celebrity). See also subtitle of this post.
- {*PRESCRIPTIVE OPINION*}, **don't** {*THING YOU DO*}
 - e.g. Parse, don't validate
 - alternative prescriptive opinion: **Never** {*THING YOU DO*}
 - see also title of this post
- {*POPULAR THING*} **IS DEAD**
 - unfortunately this is clickbait that works
 - e.g. Prompt Engineering is Dead, Big Data is Dead
 - when given at a conference for *POPULAR THING*, this is known as a Heresy Talk
 - opposite also works:
 - **The Rise of {THING}**
 - **The Unreasonable Effectiveness of {THING}**
- **Fundamentals of {THING}**
 - straightforward, people love fundamentals
 - also **How {THING} Really Works**

- You can also flip it and go for the **Advanced** {THING} tier content
- {CONVEY EXPERIENCE/SCALE} with {THING}
 - e.g. What We Learned from a Year of Building with LLMs, Insights from over **10,000** comments on "Ask HN: Who Is Hiring" using GPT-4o

There's loads more templates that work, please suggest in comments and I can add/review.

You can also browse the masters of microcopy like Steph Smith and Shaan Puri, and if you are worried about clickbait, you can see Veritasium's take or Simon Willison's Highlights.

also Andrew Chen senpai:

Understand Buzzword Metagame

Buzzwords work. Because of a mix of human psychology and algorithmic recsys. Don't avoid them, just use them judiciously. This involves the ability to read the room, which many smart people sadly cannot without significant involvement in the broader community you are trying to speak to. I don't see any way around it other than trying to authentically immerse, or consult a friendly who does.

Not all buzzwords are created equal. Databricks/Berkeley tried to push Compound AI Systems, Gartner is pushing Composite AI and Composable business — this resonates with their audience, but not more broadly.

Pick the shibboleths that work for the community you want. AI Engineer worked for reasons that are well understood now. GraphRAG is working outside of the company that coined it. These buzzwords travel, go ahead and put them in there.

Buzzwords rise and fall. Try to catch them on the rise — go harder than everyone else on the rise up and regularly test their effectiveness, such that by the time your main buzzword has tapped out you've got 2-3 others cooking.

Workshopping your titles

I write this post out of frustration, but also to capture my own thought process. I'm currently procrastinating while writing a Latent Space post.

- The original title was Why every AI Engineer needs an AI Gateway. Good, genuine opinion, load bearing. But not super memorable.
- My next iteration was **“Stop writing model routers, use an LLM Gateway”**. Ok, prescriptive, evocative. But didn't cover the other elements other than model routing. One could shorten to “Stop writing your own LLM Gateway” but that doesn't quite express what people do.
- My next one is **“It's not your job to DIY an LLM Gateway”**. No buzzword bc “your job” is implied (for a Latent Space audience), a little curiosity gap, LLM buzzword, strong opinion. But still not strong enough.
- Final one is **“LLM Gateway: The One Decision That Removes 100 AI Engineering Decisions”**. Leans on Tim Ferriss' madlbs.

It's very common for the “tail to wag the dog to wag the tail” - the title is so important to clickthrough and so impactful to the message, that you spend a bunch of time on just the title itself, because that determines the piece you will write, then write the piece, then you go back and tweak the title based on what happened.

Amateur writers think their content has 95% of the value and then spend 5% on the title. Pros will put the value of titles closer to 50% and then act like it.

You Should Break The Rules Once You Understand Human Behavior More Intuitively Than Rules Can Model

See title of this post. No better aura than intentional, conspicuous, successful rulebreaking.

the senior engineer death spiral

SUNIL PAI · 19 SEP 2026 · [SOURCE](#)

(a friend recently got a new job, a very senior role, very well paid, and pretty different from his previous gig. he was asking me how to do a 60 to 80-hour week, how to do well enough to get promoted, etc. and as we talked, I realised he was feeling a bit of imposter syndrome and really wanted to prove himself. I ended up giving him a bit of a monologue. this is more or less what I told him.)



A hand-drawn grid spiraling into a dense knot, with one loose line escaping to the edge.

okay, so this is the most common failure mode I've seen. I call it the senior engineer death spiral. it usually happens when an engineer goes into a new job, gets a promotion, or even just gets a big project at work. or they ask for a big project because they think to themselves, "oh, if I work harder, do a bigger thing, then I will be rewarded with promotions and what have you," right? like, that's the move. and the thing they do is they tell themselves they need to almost cosplay being a more senior engineer than they are. they're like, "oh, you know what, I'm going to try designing something way more ambitious."

and what happens is they start disappearing for longer periods of time. like, for two, three weeks, you won't hear anything from them. occasionally, during the standup, they'll give what I call the positive update. "yeah, things are going well, you guys. I'll have something to show you quite soon. if you have any questions, reach out." and no one usually reaches out, but they won't really have anything to show.

and in the back of their head, right, they've started the spiral already. they're telling themselves, "oh my god, I haven't really shipped anything in a while. what I need to do is work harder. I'll just work hard enough. I'll save this and no one will know." like, "in the next week, I'll do a whole month's work. and no one will know." they'll start losing sleep, all of that. and it sucks, because bro, they start getting depressed, missing meals. their relationships suffer, both work and personal. and they just start spiraling, spiraling, and it ends up in worst-case scenarios. they burn out, they need to take a month off, they get PIP'd, they get fired, they quit their jobs because they think, "you know what, this is not salvageable." and it really sucks.

and by the way, this isn't something that I've noticed only with other people. this has happened to me. and it's happened so many times that I now recognize it early enough to take a step back and fix it.

see, man, the bigger point here is that because of COVID and coding agents and remote work, things that have happened in the last five to ten years, right, you don't really have the same structure of working next to someone and being given Jira tasks, etc. you have a lot more ownership, and you are unfortunately a little more siloed. so you never want to be in a situation where people don't know what's up with you. like, they shouldn't be saying, "yeah, I don't know what Sunil's been doing," and starting to get concerned, because they might only tell you when it's too late. you want to be in a position where "Sunil doesn't shut the fuck up about what he's working on," right? again, you don't have to be obnoxious about it. you have to share your work.

I think in moments like this, the thing you need to do is actually a little counterintuitive. but you have to start from a position of assuming that everyone around you is operating in good faith, okay? like, they hired you for you. they didn't hire you for who you think you're going to be in six months. they like you for who you are right now, bro.

so the thing to do is actually to drop a level. not to try to be a higher-level engineer, but to almost drop a level and simply become the greatest teammate for a while. you're still responsible for your work, you're just trying to get moving again. you know, "I'm just here to help and support my teammates. I'm going to do bugs, the annoying things on somebody else's plate or that no one has been able to get to. do grunt work, organizational stuff, write-ups. just things to help people out."

and the reason you want to do this is you want to move away from an outcome-based mindset to momentum-based. like, you want to build a routine, because momentum is everything in this, and relationships are an even bigger thing in this. and by getting momentum going and fixing your relationships with other people, the thing you get is stamina. and that's when you realize

the real lesson, which is that big projects are not done with big efforts. it's a big marathon, just slow, steady, incremental work. forget about a weekly basis, like a daily basis, an hourly basis, to the point that it's muscle memory.

you're in automatic mode. you wake up, you make yourself coffee, you get to work, and you just do the work. and then 30 days later, you look back and you see the immense volume of work that you've done just by doing a little bit every day. because the thing you're going for is reliability, fixing your relationships. you want to take the burden off your teammates and your manager, because then, once you build that trust, they will give you bigger work. that's the move here.

remember, you are in the reputation-building business, and software is actually downstream of that. godspeed and best of luck.